

Fluidity Money (Fluid Assets)

SMART CONTRACT

Security Audit

Performed on Contracts:

ATokenInterfaces.sol
CTokenInterfaces.sol
AaveLiquidityProvider.sol
CompoundLiquidityProvider.sol
ITransferWithBeneficiary.sol
LiquidityProvider.sol
Token.sol
WorkerConfig.sol

Platform
ETH/SOL

hashlock.com.au

OCTOBER 2022

Table of Contents

Executive Summary	4
Project Context	4
Roadmap	8
The Fluidity Money Team	9
Audit scope	10
Security Rating	11
Standardised Checks	13
Intended Smart Contract Functions	15
Function List	16
Function Analysis	18
Code Quality	27
Audit Resources	27
Dependencies	27
Severity Definitions	28
Audit Findings	29
Other Project Security Considerations	33
Centralisation	34
Conclusion	35
Our Methodology	36
Disclaimers	38
About Hashlock	39
Appendix	40

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

Fluidity Money's team partnered with Hashlock to conduct a security audit of their ethereum smart contracts prior to their project's launch. Hashlock manually and proactively reviewed the code in order to ensure the protocols team and community is ready for mainnet deployment.

Project Context

[Fluidity Money](#) aims to solve one of crypto's long lasting issues and questions, "why spend crypto rather than just *hodling*?". The protocol allows users to exchange their tokens for fluid assets ~ 1:1 wrapped assets of the corresponding token. Fluidity's smart contracts, along with a *currently* centralised oracle, incentivises users with variable rewards for transacting with these fluid tokens. These rewards are designated to the periods of time they were earned measured by block number, and are earned by fluidity investing the underlying tokens. Users can then exchange these tokens for the original underlying token when they please.

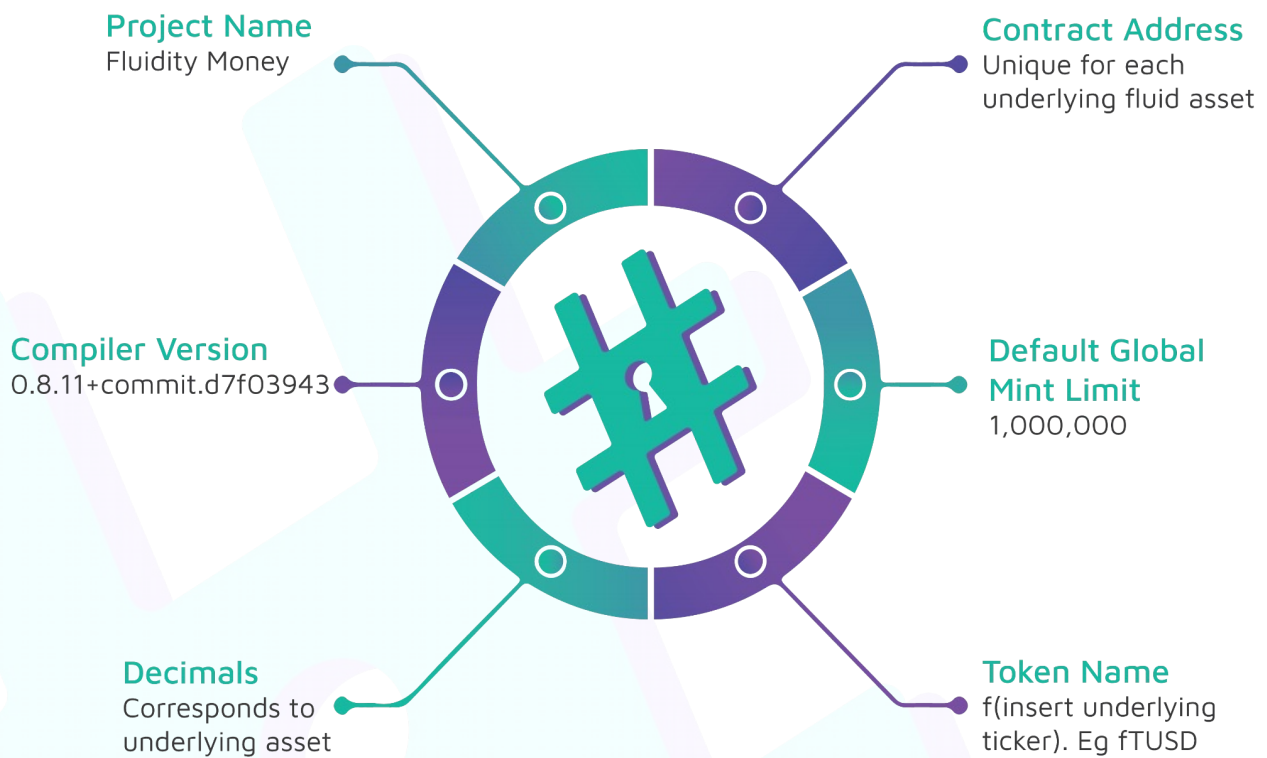
Token Name: Fluidity Tokens. [Syntax is f\(insert underlying ticker\)](#). Eg [fTUSD](#).

Compiler Version: 0.8.11+commit.d7f03943

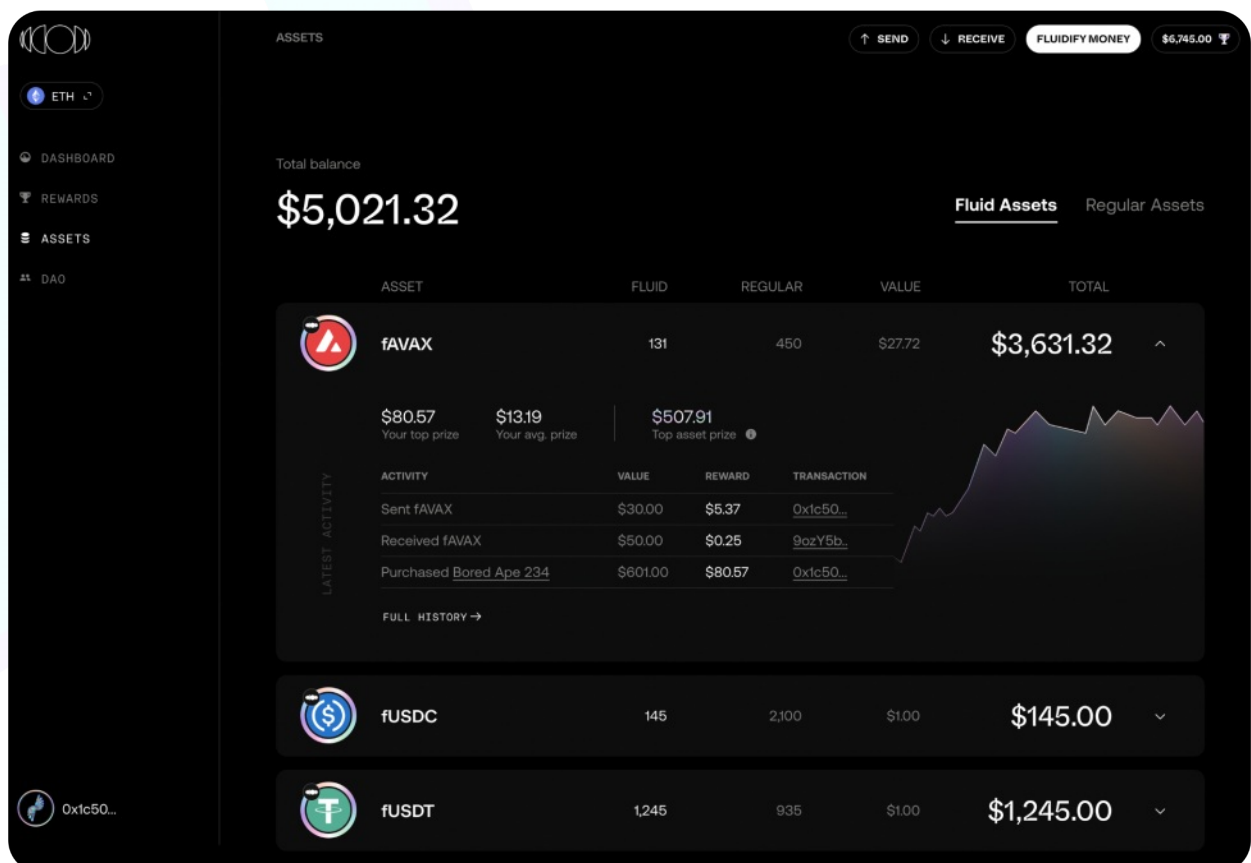
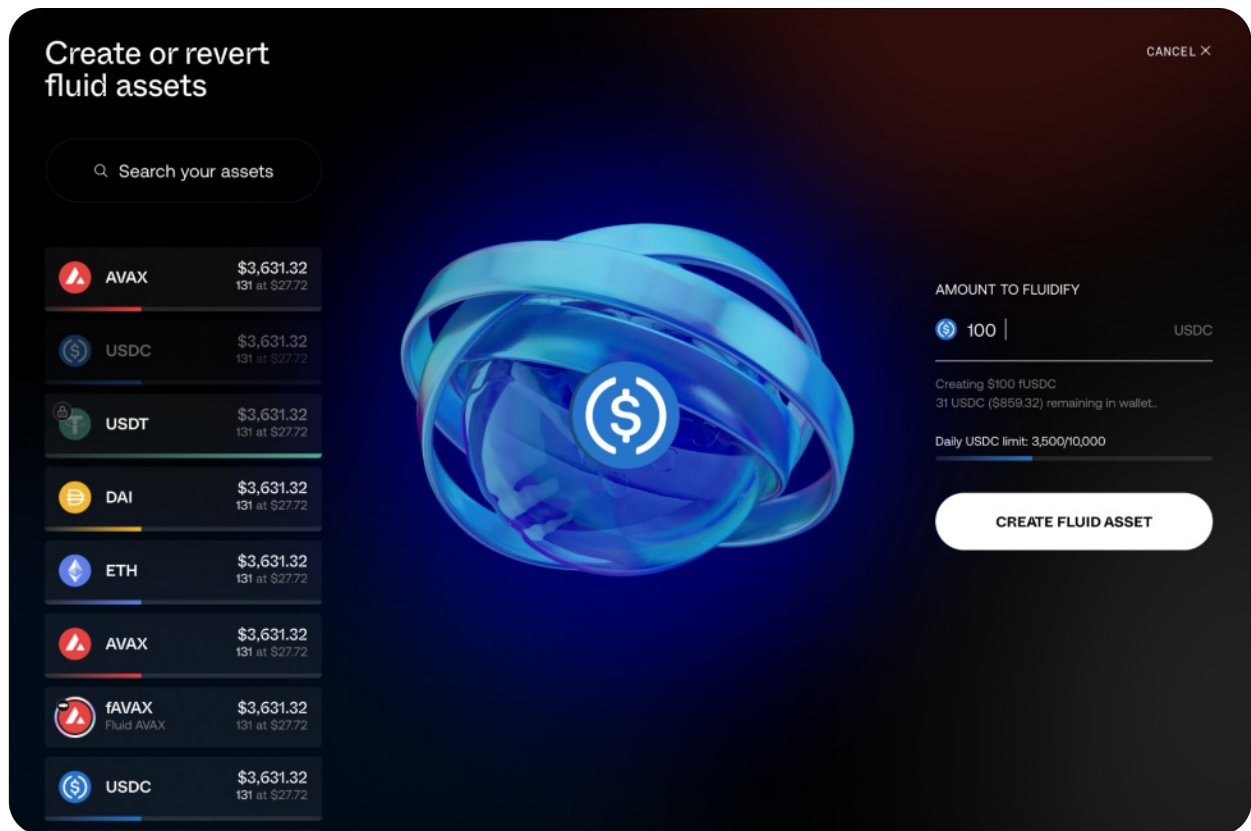
Decimals: Varies depending on what token is being wrapped.

Logo:

fluidity

Visualised Context:

Platform Visualisation (Pre release):





ETH

DASHBOARD

REWARDS

ASSETS

DAO

REWARDS

↑ SEND

↓ RECEIVE

FLUIDIFY MONEY

\$6,745.00



Unclaimed fluid rewards

\$6,745.00

VIEW BREAKDOWN

Auto-claims

Rewards will be claimed automatically, without fees when market volume is reached. Claiming before this time will incur instant-claim fees stated below.

Instant-claim fees

Network fee	\$0.002 FUSDC
Gas fee	\$0.002 FUSDC

Reward Performance

All time

Last week

Last month

This year

Total claimed yield

\$29,645.00

Highest performer

 fAVAX

Total prize pool

\$678,120.00

Fluid pairs

8

1-12 of 120 reward transactions

All

DEX

NFT

DeFi

0x1c50...

ACTIVITY

VALUE

REWARD

ACCOUNT

TIME



CANCEL X



Congrats! you've claimed

\$6,745.00 USD

The funds have been added to your wallet.

Network fee	\$0.002 FUSDC
Gas fee	\$0.002 FUSDC

GO TO ASSETS

SHARE

REWARDS HISTORY →



Roadmap

The project features a traditional rollout plan as well as a decentralisation roadmap. Due to the random nature of the incentivisation structure, their plan to launch with a higher level of centralisation, specifically in their oracle, decreases the risk of emergency.

As of this report being written, the project plans to launch Q4 2022 on Ethereum first, and Solana later.

Their rollout plan is as follows (content from their [roadmap](#)):

1. Mainnet Launch ~ Approx Q4, 2022.
2. Fluidity DAO ~ DAO Framework and Fluidity Token
3. Utility Mining & Fluidity Wars ~ Utility Gauges, Governance and protocol/user participation
4. The Incentive Layer ~ Fluidity as a chain agnostic incentive middle layer
5. Security and Decentralisation ~ Randomness, System Architecture
6. Ambassador program and Fluidity Ecosystem Fund
7. Web2 - Web3 and retail: Bridging adoption
8. (🌊, 💎) Join The Movement

Their official documentation also lists some of their internal goals/ambitions for 2023.

- \$20M Wrapped
- Retail onboarding through fiat onramp
- Retail adoption of crypto on some use cases - e.g. instant checkout technologies
- NFT and Digital Art accessibility promotion through Fluid Assets.

Decentralisation plan (content reference [here](#)):

(WIP)

- Fluidity worker/contract architecture - centralised, little trust in the randomness generation <- we are here.
- Distributed worker/contract architecture - publicly sourced randomness from a multiparty computation with leader election, competition with batching powered by members of the community.
- 100% decentralised architecture including randomness generation, zero dependency on Fluidity services.

The Fluidity Money Team

Title	Name	Photo	Links
Project Founder	Alexander Baigent		Linkedin
Project Founder	Shahmeer Chaudhry		Linkedin

Audit scope

We at Hashlock audited the solidity code within the protocol, however note that their project also features rust, goLang, and a centralised oracle that can also be a source of vulnerabilities.

The solidity code however is the foundation of the project's security on the ethereum network.

Description	Project Review and Security Analysis Report for Fluidity Money Protocol Smart Contracts and other factors.
Platform	Ethereum / Solidity
Audit Date	October 7th, 2022
Contract 1	Token.sol
Contract 1 MD5 Hash	7a5ae54d55b15f6569d8f2b9941bd1cc
Contract 2	WorkerConfig.sol
Contract 2 MD5 Hash	4830c696d0636fd2cb5a7e321c9dcbf9
Contract 3	LiquidityProvider.sol
Contract 3 MD5 Hash	fc6639a747956d151186ec94f9eeabdb
Contract 4	ITransferWithBeneficiary.sol
Contract 4 MD5 Hash	99549bcb38e76ce734607942dfecd0b9
Contract 5	CompoundLiquidityProvider.sol
Contract 5 MD5 Hash	31c574eed518b5fe41932f5d29388352
Contract 6	AaveLiquidityProvider.sol
Contract 6 MD5 Hash	c49bebbba32d31107f1c6f2bfd7a604fe

Security Rating

After our audit and analysis, we found the smart contracts to be **"Secure"**. The primary contracts are Token.sol, WorkerConfig.sol and LiquidityProvider.sol, and all follow simple logic, with correct and detailed ordering. They use a series of interfaces for operations such as liquidity pool interactions, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for fully decentralised, highly secure projects with little owner influence.

Our primary area of concern was to try to find ways to take ownership of the contract via the `_operator`, `_emergencyCouncil` or `_oracle` address state variables. The contract does not use Ownable.sol, however it uses the same 'require' logic via `msg.sender` validation.

There does not seem to be any arithmetic or logic errors that affect security or functionality.

All issues uncovered during automated analysis were manually reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

Our team identified no impactful code vulnerabilities, however there are some security factors to be aware of in relation to the centralised oracle and owner dependencies, as well as a few low level / informational code inefficiencies. The code has been well tested, has sound security / emergency features and appears ready to launch.

Caution: *Hashlock's audit's do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Reviewed
	Outdated Solidity Version	Passed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Passed
	Lack of check on input parameters	Reviewed
	Function input parameters check bypass	Passed
	Correct Access control	Passed
	Built in emergency features	Passed
	Correct event logs	Passed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Passed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed
	High consumption 'storage' storage	Passed

	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Overall Audit Result: PASSED

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Contract 1 Token.sol - Adds underlying assets to liquidity pool and mints the user fluid assets - Has emergency / security features such as mint limits and emergency mode. - Access controls / oracle addresses can be updated.	Contract achieves this functionality.
Contract 2 WorkerConfig.sol - Oracle (worker config) can enable emergency mode and disable it. - Oracle address is set by the worker and signed off on by the multisig.	Contract achieves this functionality.
Contract 3 LiquidityProvider.sol - Interface for protocol to add underlying assets to pools and identify the traits of that asset.	Contract achieves this functionality.
Contract 4 ITransferWithBeneficiary.sol - Interface to allow a 'signer' to pay tokens giving benefits to the beneficiary.	Contract achieves this functionality.
Contract 5 CompoundLiquidityProvider.sol - Operator can add pools and transfer in and out of that pool.	Contract achieves this functionality.
Contract 6 AaveLiquidityProvider.sol - Operator can add pools and transfer in and out of that pool.	Contract achieves this functionality.

Function List

AaveLiquidityProvider.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	init	external	Function input parameters lack of check	Refer Audit Findings
3	addToPool	external	Passed	No Issue
4	takeFromPool	external	Passed	No Issue
5	totalPoolAmount	external	Passed	No Issue

CompoundLiquidityProvider.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	init	external	Function input parameters lack of check,	Refer Audit Findings
3	addToPool	external	Passed	No Issue
4	takeFromPool	external	Passed	No Issue
5	totalPoolAmount	external	Passed	No Issue

Token.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	init	write	Passed	No Issue
3	setRestrictions	write	Passed	No Issue
4	updateOperator	write	Passed	No Issue
5	noEmergencyMode	read	Passed	No Issue
6	oracle	read	Passed	No Issue
7	enableEmergencyMode	write	Passed	No Issue
8	disableEmergencyMode	write	Passed	No Issue
9	updateWorkerConfig	write	Passed	No Issue
10	updateRewardQuarantineThresh old	write	Passed	No Issue
11	updateMintLimits	write	Passed	No Issue
12	enableMintLimits	write	Passed	No Issue
13	erc20In	write	Passed	No Issue
14	erc20Out	write	Passed	No Issue
15	rewardPoolAmount	write	Passed	No Issue

16	rewardFromPool	internal	Passed	No Issue
17	rewardInternal	internal	Passed	No Issue
18	batchReward	write	Passed	No Issue
19	unlockReward	write	Passed	No Issue
20	manualReward	external	Passed	No Issue
21	recover	internal	Passed	No Issue
22	name	read	Passed	No Issue
23	symbol	read	Passed	No Issue
24	decimals	read	Passed	No Issue
25	totalSupply	read	Passed	No Issue
26	balanceOf	read	Passed	No Issue
27	transfer	write	Passed	No Issue
28	allowance	read	Passed	No Issue
29	approve	write	Passed	No Issue
30	transferFrom	write	Passed	No Issue
31	transferWithBeneficiary	external	Two transfers can be merged into one	Refer Audit Findings
32	increaseAllowance	write	Passed	No Issue
33	decreaseAllowance	write	Passed	No Issue
34	_transfer	internal	Passed	No Issue
35	_mint	internal	Passed	No Issue
36	_burn	internal	Passed	No Issue
37	_approve	internal	Passed	No Issue
38	_spendAllowance	internal	Passed	No Issue

WorkerConfig.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	init	write	Function input parameters lack of check	Refer Audit Findings
3	noGlobalEmergency	read	Passed	No Issue
4	updateOracles	write	Passed	No Issue
5	getWorkerAddress	read	Passed	No Issue
6	getWorkerAddress	read	Passed	No Issue
7	enableEmergencyMode	write	Passed	No Issue
8	disableEmergencyMode	write	Passed	No Issue

Function Analysis

Fluidity Money's protocol can be clearly understood by analysing their Token.sol contract. Each underlying asset (fluid asset) will have its own Token.sol contract address. The function of this contract is to take selected ERC20 assets, invest them in a pool, and mint the user fluid assets in a 1-1 ratio, with the same decimals as the underlying token. This contract has functions to pay users when the oracle assigns them a reward, and to burn fluid assets in exchange for the underlying tokens. Below we isolate key arguments to analyse. We did not display entire function code where it was not necessary to increase readability.

Function init:

```
Token.sol  
175         require(version_ == 0, "contract is already initialised");  
176         version_ = 1;
```

This can be seen as a manual constructor, as it can only be called once for each deployment due to the argument. This is needed due to the interoperability with beaconproxy/upgradeablebeaconproxy.

Function updateOperator:

```
Token.sol  
239         function updateOperator(address newOperator) public {  
240             require(msg.sender == operator_, "only operator can use this function!");  
241         }
```

Uses the same logic seen in Ownable.sol for verification and security. Standard function.

Function noEmergencyMode:

```
Token.sol  
  
247     function noEmergencyMode() public view returns (bool) {  
248         return noEmergencyMode_  
249             && workerConfig_.noGlobalEmergency();  
250     }  
251
```

Fluidity has followed best practices by implementing an emergency mode to minimise vulnerability. This is a view function so no risk.

Function oracle:

```
Token.sol  
  
256     function oracle() public view returns (address) {  
257         return workerConfig_.getWorkerAddress(address(this));  
258     }
```

Shows the address of the oracle, which can be changed. View function no risk.

Function enableEmergencyMode:

```
Token.sol

264     function enableEmergencyMode() public {
265         require(
266             msg.sender == operator_ ||
267             msg.sender == emergencyCouncil_ ||
268             msg.sender == oracle(),
269             "can't enable emergency mode!"
270         );
271
272         noEmergencyMode_ = false;
273
274         workerConfig_ = WorkerConfig(address(0));
275
276         emit Emergency(true);
277     }
```

This is a great function to have. Only the operator, oracle or emergency council can call it. Only possible risk here is if a contract or person was able to enable emergency mode by gaining ownership or passing another criteria.

Function disableEmergencyMode:

```
Token.sol

283     function disableEmergencyMode(address _workerConfig) public {
284         require(msg.sender == operator_, "only the operator account can use this");
```

The emergencyCouncil can enable but not disable emergency mode, which increases security but places more importance on the operator wallet address.

Function updateWorkerConfig:

```

Token.sol

293     function updateWorkerConfig(address _workerConfig) public {
294         require(msg.sender == operator_, "only the operator account can use this");
295
296         workerConfig_ = WorkerConfig(_workerConfig);
297
298         emit WorkerConfigUpdated(_workerConfig);
299     }

```

This function has no comments. This updates the oracle address.

Function updateRewardQuarantineThreshold:

```

Token.sol

302     function updateRewardQuarantineThreshold(uint _maxUncheckedReward) public {
303         require(msg.sender == operator_, "only the operator account can use this");
304
305         maxUncheckedReward_ = _maxUncheckedReward;
306
307         emit RewardQuarantineThresholdUpdated(_maxUncheckedReward);
308     }

```

No real risk here as long as no one takes ownership (becomes operator) of the contract.

Function updateMintLimits:

```

Token.sol

311     function updateMintLimits(uint global, uint user) public {
312         require(noEmergencyMode(), "emergency mode!");
313         require(msg.sender == oracle(), "only the oracle account can use this");
314
315         remainingGlobalMint_ = global;
316         userMintLimit_ = user;
317         userMintResetBlock_ = block.number;
318
319         emit GlobalMintLimitChanged(global);
320         emit UserMintLimitChanged(userMintLimit_);
321     }

```

Keep in mind that the default global mint limit is set to 100,000 by default. This is great to have as a security feature.

Function enableMintLimits:

```

324     function enableMintLimits(bool enable) public {
325         require(msg.sender == operator_, "only the operator account can use this");
326
327         mintLimitsEnabled_ = enable;
328
329         emit MintLimitsStateChanged(enable);
330     }

```

Standard and simple logic.

Function erc20In:

This function is ordered correctly. It makes checks first via require statements, takes the erc20 tokens from the user first, ensures they are correctly placed in the pool, before minting the user fluid assets. See the logic after checks are made:

```

370         // take underlying tokens from the user
371         uint originalBalance = pool_.underlying().balanceOf(address(this));
372         pool_.underlying().safeTransferFrom(msg.sender, address(this), amount);
373         uint finalBalance = pool_.underlying().balanceOf(address(this));
374
375         // ensure the token is behaving
376         require(finalBalance > originalBalance, "token balance decreased after transfer");
377         uint realAmount = finalBalance - originalBalance;
378
379         // add the tokens to our compound pool
380         pool_.underlying().safeTransfer(address(pool_), realAmount);
381         pool_.addToPool(realAmount);
382
383         // give the user fluid tokens
384         _mint(msg.sender, realAmount);
385         emit MintFluid(msg.sender, realAmount);
386         return realAmount;

```

What we looked for here was possible errors adding the tokens to the pool. Another check before minting fluid assets could act as an extra layer of security but not necessary.

Function `erc20Out`:

The burn function has the necessary checks in order to make this operation secure, ensuring that the address holds enough fluid assets ect. Correct order is used to avoid reentrancy.

```
Token.sol

394     function erc20Out(uint amount) public {
395         // take the user's fluid tokens
396         _burn(msg.sender, amount);
397
398         // give them erc20
399         pool_.takeFromPool(amount);
400         pool_.underlying_().safeTransfer(msg.sender, amount);
401         emit BurnFluid(msg.sender, amount);
402     }
```

Function `rewardPoolAmount`:

```
Token.sol

409     function rewardPoolAmount() public returns (uint) {
410         uint totalAmount = pool_.totalPoolAmount();
411         uint totalFluid = totalSupply();
412         require(totalAmount >= totalFluid, "balance is less than total supply");
413         return totalAmount - totalFluid;
414     }
```

Returns pool amount, no risk.

Function rewardFromPool:

This function is where we start to see rewards being quarantined. This affects centralisation, but is in the roadmap to change over time.

```

Token.sol

424 function rewardFromPool(uint256 firstBlock, uint256 lastBlock, address winner, uint256 amount) internal {
425     require(noEmergencyMode(), "emergency mode!");
426
427     if (amount > maxUncheckedReward_) {
428         // quarantine the reward
429         emit BlockedReward(winner, amount, firstBlock, lastBlock);
430
431         blockedRewards_[winner] += amount;
432
433         return;
434     }

```

Function rewardInternal:

Internal function with correct logic, no risk. This is the function the contract calls internally to pay rewards.

```

Token.sol

439 function rewardInternal(uint256 firstBlock, uint256 lastBlock, address winner, uint256 amount) internal {
440     require(noEmergencyMode(), "emergency mode!");
441
442     // mint some fluid tokens from the interest we've accrued
443     emit Reward(winner, amount, firstBlock, lastBlock);
444
445     _mint(winner, amount);
446 }

```

Function batchReward:

This function pays rewards in groups, it uses somewhat complicated logic, but uses the required checks, and updates variables before minting rewards.

```

Token.sol

453 function batchReward(winner[] memory rewards, uint firstBlock, uint lastBlock) public {
454     require(noEmergencyMode(), "emergency mode!");
455     require(msg.sender == oracle(), "only the oracle account can use this");
456
457     uint poolAmount = rewardPoolAmount();

```

Function unlockReward:

This function releases quarantined rewards. Logic is secure, however it relies on the operator address.

```

Token.sol

487     function unlockReward(address user, uint amount, bool payout, uint firstBlock, uint lastBlock) public {
488         require(noEmergencyMode(), "emergency mode!");
489         require(msg.sender == operator_, "only the operator account can use this");
490
491         require(blockedRewards_[user] >= amount,
492             "trying to unblock more than the user has blocked");
493
494         blockedRewards_[user] -= amount;
495
496         if (payout) {
497             rewardInternal(firstBlock, lastBlock, user, amount);
498         }
499     }

```

Function manualReward:

Allows users to manually call rewards to be paid from the frontend, paying their own gas. See the amount of checks before minting.

```

Token.sol

534     // ECDSA verification
535     require(recover(hash, sig) == oracle(), "invalid rng signature");
536
537     // now reward the user
538
539     require(contractAddress == address(this), "payload is for the wrong contract");
540     require(chainid == block.chainid, "payload is for the wrong chain");
541
542     // user decided to frontrun
543     require(
544         firstBlock > lastRewardedBlock_,
545         "reward already given for part of this range"
546     );
547
548     for (uint i = firstBlock; i <= lastBlock; i++) {
549         require(manualRewardedBlocks_[winner][i] == 0, "reward already given for part of this range");
550         manualRewardedBlocks_[winner][i] = BLOCK_REWARDED;
551     }
552
553     manualRewardDebt_[winner] += winAmount;
554
555     require(rewardPoolAmount() >= winAmount, "reward pool empty");
556
557     rewardFromPool(firstBlock, lastBlock, winner, winAmount);
558 }

```

Function recover:

Internal function with correct logic, no risk.

```

560      /**
561       * @dev Ecrecover with checks for signature malleability
562       * @dev adapted from openzeppelin's ECDSA library
563       */
564      function recover(
565          bytes32 hash,
566          bytes memory signature
567      ) internal pure returns (address) {
568          require(signature.length == 65, "invalid rng format (length)");
569          bytes32 r;
570          bytes32 s;
571          uint8 v;
572          // ecrecover takes the signature parameters, and the only way to get them
573          // currently is to use assembly.
574          /// @solidity memory-safe-assembly
575          assembly {
576              r := mload(add(signature, 0x20))
577              s := mload(add(signature, 0x40))
578              v := byte(0, mload(add(signature, 0x60)))
579          }

```

Remaining functions:

The remaining functions in Token.sol are from Open Zeppelin's ERC20 Implementation.

Code Quality

This audit scope involves the solidity smart contracts of the Fluidity Money project, as outlined in the Audit Scope section. All contracts, libraries and interfaces follow standard best practices and avoid unnecessary complexity that increases the likelihood of exploitation.

All code is well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Fluidity Money Protocol's smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are very helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
Critical	Critical vulnerabilities are of immediate concern, and allow hackers or malicious code to steal funds, take ownership of the contract, or other devastating actions.
High	High vulnerabilities are not attacked as easily, but still have significant implications on the protocol.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Informational	These are mostly preferred factors that won't affect security.

Audit Findings

Critical

No critical vulnerabilities were found.

High

No high-level vulnerabilities were found.

Medium

No medium vulnerabilities were found.

Low

1) Transfer inefficiency that can be merged into one transfer: - [Token.sol](#)

```
Token.sol

642     function transferWithBeneficiary(
643         address token,
644         uint256 amount,
645         address beneficiary,
646         uint64 data
647     ) external override returns (bool) {
648         bool rc;
649
650         rc = Token(token).transferFrom(msg.sender, address(this), amount);
651         if (!rc) return false;
652
653         rc = Token(token).transfer(beneficiary, amount);
654
655         return rc;
656     }
```

This code involves two transfers. One transfer from the caller to the contract, and the second from the contract to the beneficiary.

Resolution: We suggest one transfer from caller to beneficiary to save gas.

Status: Resolved.

Advised that the reason for logic is accurate by following current Meson's specifications.

(2) Lack of best practice check on init() functions:

CompoundLiquidityProvider.sol

- init = compoundToken.

```
CompoundLiquidityProvider.sol

34     function init(
35         address compoundToken,
36         address owner
37     ) external {
38         require(version_ == 0, "contract is already initialized");
39         version_ = 1;
40
41         owner_ = owner;
42
43         compoundToken_ = CErc20Interface(compoundToken);
44
45         underlying_ = IERC20(compoundToken_.underlying());
46         underlying_.safeApprove(address(compoundToken_), type(uint).max);
47     }
```

WorkerConfig.sol

- init = _operator, _emergencyCouncil.

```

WorkerConfig.sol

47     function init(
48         address _operator,
49         address _emergencyCouncil
50     ) public {
51         require(version_ == 0, "contract is already initialised");
52         version_ = 1;
53
54         operator_ = _operator;
55
56         emergencyCouncil_ = _emergencyCouncil;
57
58         noGlobalEmergency_ = true;
59     }

```

AaveLiquidityProvider.sol

- init = addressProvider, aToken.

```

AaveLiquidityProvider.sol

36     function init(
37         address addressProvider,
38         address aToken,
39         address owner
40     ) external {
41         require(version_ == 0, "contract is already initialized");
42         version_ = 1;
43
44         owner_ = owner;
45
46         lendingPoolAddresses_ = LendingPoolAddressesProviderInterface(addressProvider);
47         aToken_ = ATokenInterface(aToken);
48
49         underlying_ = IERC20(aToken_.UNDERLYING_ASSET_ADDRESS());
50     }

```

Resolution: We advise to check that address is not 0, and integer type variables are greater than 0. In use however these contracts will be initialised by operators / fluidity team, so this issue is really just a code best practice.

Status: Acknowledged.

#Hashlock.

Hashlock Pty Ltd

(3) The protocol currently does not contain a registry contract. Therefore, all instances for each underlying token / fluid asset are not recorded. Instances should only be deployable from a factory/registry contract that records each instance.

Status: Acknowledged.

Very Low / Informational / Best practices:

(1) SPDX licence identifier missing: [ATokenInterfaces.sol](#), [CTokenInterfaces.sol](#)

SPDX licence identifier is not provided.

Resolution: We suggest adding SPDX licence identifier at the top of the code.

Status: Acknowledged.

Informational:

The contracts lack ownable.sol, but use the same logic.

The contracts seem to be intended to be deployed multiple times, one for each underlying asset.

The contracts do not use the most recent version of solidity, and versioning is not consistent in ITransferWithBeneficiary.sol.

The contracts don't use a constructor, they use a manual init() function due to the interplay between beaconproxy/upgradeablebeaconproxy.

Status: Acknowledged.

Other Project Security Considerations

As mentioned throughout this report, there are several other factors of this protocol that must be considered when analysing the security of the entire fluidity ecosystem. These were not directly within the code audit scope, however please be mindful of the following:

- Private keys of oracles, operators and emergency councils.
- Reward quarantines manually released by the fluidity money team.
- Centralised oracle that decides payouts.
- Golang, rust code and the Fluidity Money solana implementation.

The Fluidity Money team have been extremely transparent about their security measures and have provided an abundance of documentation [here](#).

Centralisation

Please see the project's decentralisation roadmap [here](#):

As it stands prior to launch, this project will value security over decentralisation. They are using a centralised oracle, with a centralised process to quarantine and release large rewards manually.

The owner executable functions within the protocol increase security but depend highly on private key security, as do the worker config (oracle) keys, ect.

The project plans to eventually be a self-sufficient decentralised protocol, and its code is built from the foundation to allow for future automation.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Fluidity Money protocol seems to have a sound and well tested code base for its EVM implementation. Overall, most of the code is correctly ordered and follows industry best practices. There is also an evident focus on built in emergency procedures.

The project does sacrifice decentralisation for security, as there is a large amount of access control, as well as the manual release of rewards. They are also using a centralised oracle to decide winners and prizes. The project aims to increase decentralisation after a smooth and secure launch.

The developers have used atypical methods in some sections to better suit their goal, such as a manual init function rather than a constructor, to allow for multiple iterations of the protocol to exist for each underlying asset, rather than holding them all in one smart contract, however this is to allow for complex custom operability. All of these methods have been heavily reviewed by our team.

Factors not part of this audit scope were the oracle itself, the project's goolang or rust code, or any solana implementations, and our security review does not account for these variables.

Hashlock believes Fluidity Moneys solidity code base is ready for deployment.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

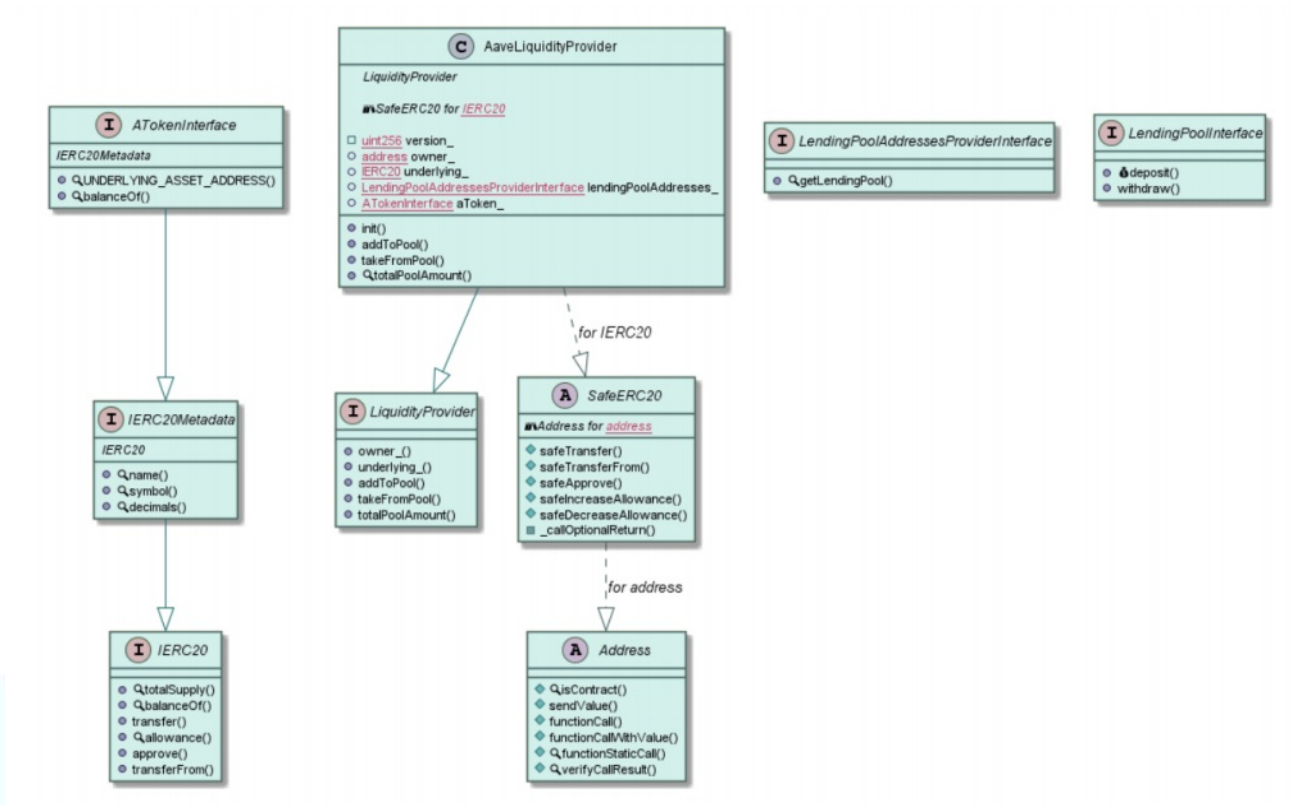
Contact: info@hashlock.com.au



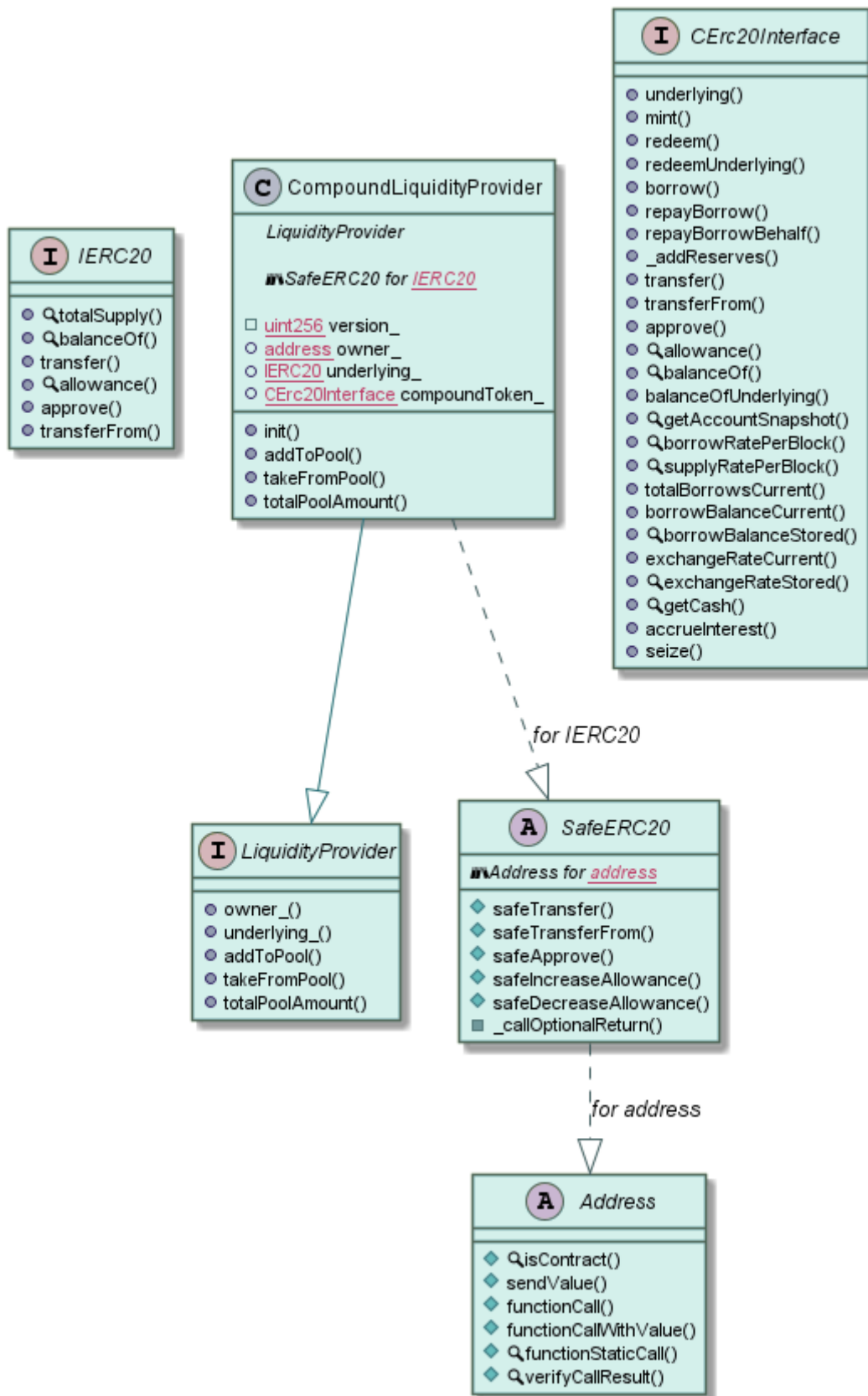
Appendix

Code Flow Diagram - Fluidity Money Protocol

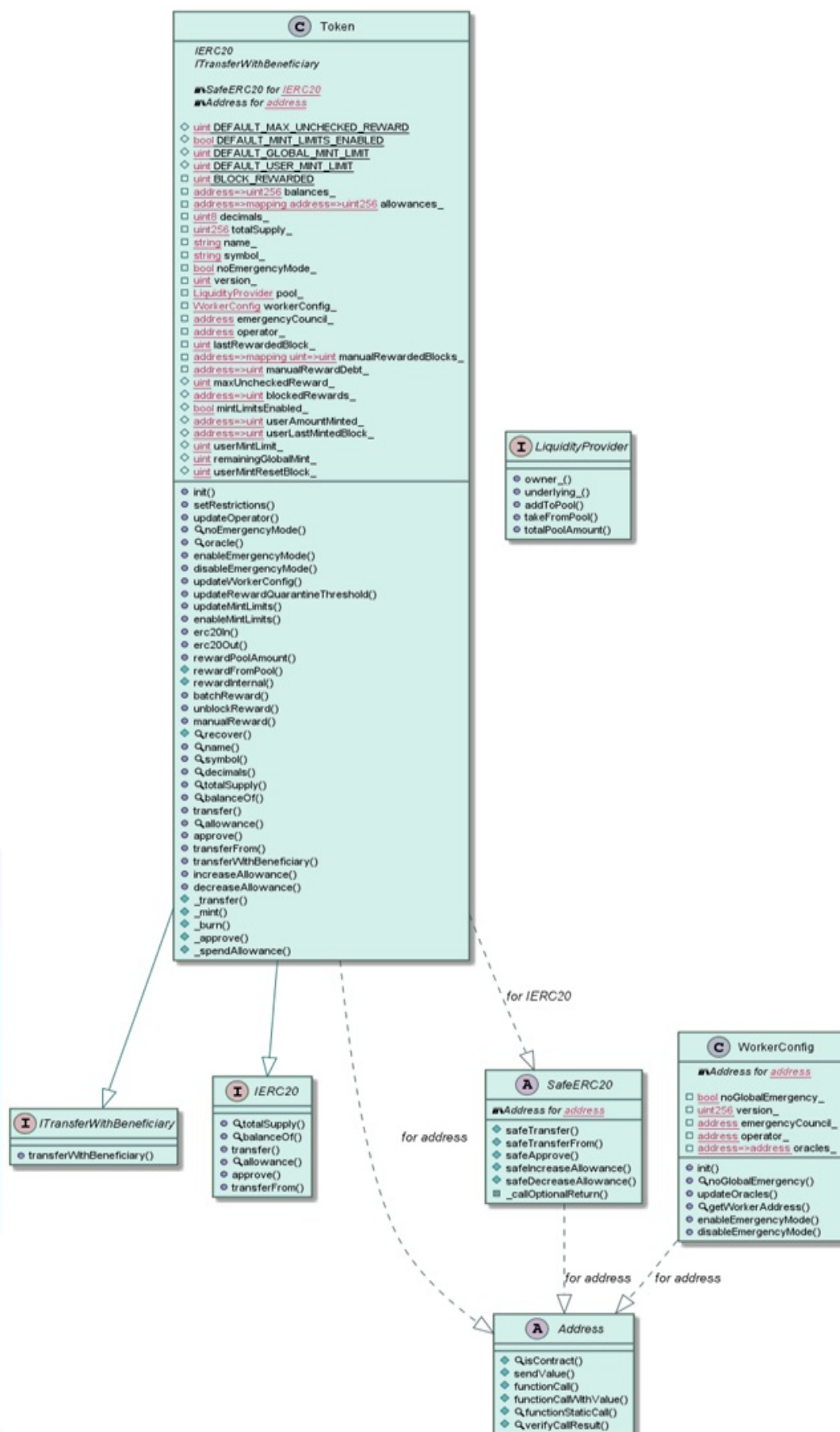
AaveLiquidityProvider Diagram



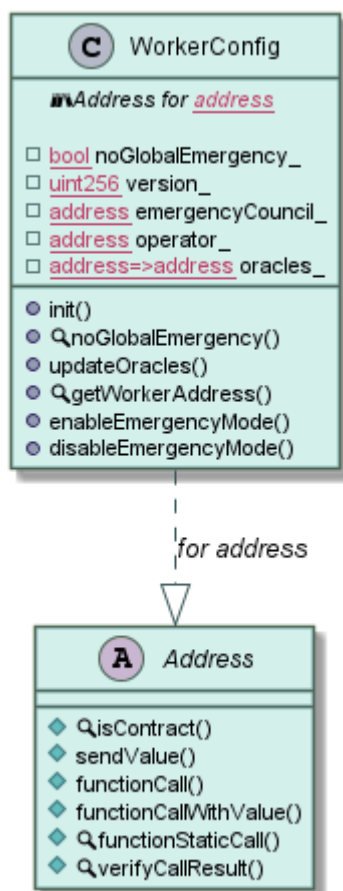
CompoundLiquidityProvider Diagram



Token Diagram



WorkerConfig Diagram



Slither Results Log

Slither log >> AaveLiquidityProvider.sol

```
INFO:Detectors:
AaveLiquidityProvider.init(address,address,address).owner (AaveLiquidityProvider.sol#479) lacks a zero-check on :
- owner = owner (AaveLiquidityProvider.sol#484)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Address.verifyCallResult(bool,bytes,string) (AaveLiquidityProvider.sol#311-331) uses assembly
- INLINE ASM (AaveLiquidityProvider.sol#323-326)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Address.functionCall(address,bytes) (AaveLiquidityProvider.sol#195-197) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256) (AaveLiquidityProvider.sol#224-230) is never used and should be removed
Address.functionStaticCall(address,bytes) (AaveLiquidityProvider.sol#257-259) is never used and should be removed
Address.functionStaticCall(address,bytes,string) (AaveLiquidityProvider.sol#267-276) is never used and should be removed
Address.sendValue(address,uint256) (AaveLiquidityProvider.sol#170-175) is never used and should be removed
SafeERC20.safeDecreaseAllowance(IERC20,address,uint256) (AaveLiquidityProvider.sol#386-397) is never used and should be removed
SafeERC20.safeIncreaseAllowance(IERC20,address,uint256) (AaveLiquidityProvider.sol#377-384) is never used and should be removed
SafeERC20.safeTransferFrom(IERC20,address,address,uint256) (AaveLiquidityProvider.sol#346-353) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code
INFO:Detectors:
Pragma version^0.8.4 (AaveLiquidityProvider.sol#7) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Low level call in Address.sendValue(address,uint256) (AaveLiquidityProvider.sol#170-175):
- (success) = recipient.call{value: amount}() (AaveLiquidityProvider.sol#173)
Low level call in Address.functionCallWithValue(address,bytes,uint256,string) (AaveLiquidityProvider.sol#238-249):
- (success,returndata) = target.call{value: value}(data) (AaveLiquidityProvider.sol#247)
Low level call in Address.functionStaticCall(address,bytes,string) (AaveLiquidityProvider.sol#267-276):
- (success,returndata) = target.staticcall(data) (AaveLiquidityProvider.sol#274)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Detectors:
Function ATokenInterface.UNDERLYING_ASSET_ADDRESS() (AaveLiquidityProvider.sol#441) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Slither:AaveLiquidityProvider.sol analyzed (9 contracts with 75 detectors), 16 result(s) found
INFO:Slither:Use https://crytic.io/ to get access to additional detectors and Github integration
```

Slither log >> CompoundLiquidityProvider.sol

```
INFO:Detectors:
CompoundLiquidityProvider.init(address,address).owner (CompoundLiquidityProvider.sol#479) lacks a zero-check on :
- owner = owner (CompoundLiquidityProvider.sol#484)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Address.verifyCallResult(bool,bytes,string) (CompoundLiquidityProvider.sol#310-330) uses assembly
- INLINE ASM (CompoundLiquidityProvider.sol#322-325)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Address.functionCall(address,bytes) (CompoundLiquidityProvider.sol#194-196) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256) (CompoundLiquidityProvider.sol#223-229) is never used and should be removed
Address.functionStaticCall(address,bytes) (CompoundLiquidityProvider.sol#256-258) is never used and should be removed
Address.functionStaticCall(address,bytes,string) (CompoundLiquidityProvider.sol#266-275) is never used and should be removed
Address.sendValue(address,uint256) (CompoundLiquidityProvider.sol#169-174) is never used and should be removed
SafeERC20.safeDecreaseAllowance(IERC20,address,uint256) (CompoundLiquidityProvider.sol#385-396) is never used and should be removed
SafeERC20.safeIncreaseAllowance(IERC20,address,uint256) (CompoundLiquidityProvider.sol#376-383) is never used and should be removed
SafeERC20.safeTransferFrom(IERC20,address,address,uint256) (CompoundLiquidityProvider.sol#345-352) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code
INFO:Detectors:
Pragma version^0.8.4 (CompoundLiquidityProvider.sol#7) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Low level call in Address.sendValue(address,uint256) (CompoundLiquidityProvider.sol#169-174):
- (success) = recipient.call{value: amount}() (CompoundLiquidityProvider.sol#172)
Low level call in Address.functionCallWithValue(address,bytes,uint256,string) (CompoundLiquidityProvider.sol#237-248):
- (success,returndata) = target.call{value: value}(data) (CompoundLiquidityProvider.sol#246)
Low level call in Address.functionStaticCall(address,bytes,string) (CompoundLiquidityProvider.sol#266-275):
- (success,returndata) = target.staticcall(data) (CompoundLiquidityProvider.sol#273)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
```

```
INFO:Detectors:
Function CERC20Interface.addReserves(uint256) (CompoundLiquidityProvider.sol#432) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Slither:CompoundLiquidityProvider.sol analyzed (6 contracts with 75 detectors), 16 result(s) found
INFO:Slither:Use https://crytic.io/ to get access to additional detectors and Github integration
```

Slither log >> Token.sol

```

- emergencyCouncil_ = _emergencyCouncil (Token.sol#159)
Token.init(address,uint8,string,string,address,address,address)._operator (Token.sol#375) lacks a zero-check on :
- operator_ = _operator (Token.sol#381)
Token.init(address,uint8,string,string,address,address,address)._emergencyCouncil (Token.sol#374) lacks a zero-check on :
- emergencyCouncil_ = _emergencyCouncil (Token.sol#383)
Token.updateOperator(address).newOperator (Token.sol#425) lacks a zero-check on :
- operator_ = newOperator (Token.sol#428)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Reentrancy in Token.batchReward(Winner[],uint256,uint256) (Token.sol#580-601):
  External calls:
  - poolAmount = rewardPoolAmount() (Token.sol#584)
  - totalAmount = pool_.totalPoolAmount() (Token.sol#552)
  State variables written after the call(s):
  - rewardFromPool(firstBlock,lastBlock,winner.winner,winner.amount) (Token.sol#599)
  - balances_[account] += amount (Token.sol#774)
  - rewardFromPool(firstBlock,lastBlock,winner.winner,winner.amount) (Token.sol#599)
  - blockedRewards_[winner] += amount (Token.sol#564)
  - lastRewardedBlock_ = lastBlock (Token.sol#586)
  - manualRewardDebt_[winner.winner] = 0 (Token.sol#593)
Reentrancy in Token.erc20In(uint256) (Token.sol#503-541):
  External calls:
  - originalBalance = pool_.underlying().balanceOf(address(this)) (Token.sol#528)
  - pool_.underlying().safeTransferFrom(msg.sender,address(this),amount) (Token.sol#529)
  - finalBalance = pool_.underlying().balanceOf(address(this)) (Token.sol#530)
  - pool_.underlying().safeTransfer(address(pool_),realAmount) (Token.sol#535)
  - pool_.addToPool(realAmount) (Token.sol#536)
  State variables written after the call(s):
  - _mint(msg.sender,realAmount) (Token.sol#538)
  - balances_[account] += amount (Token.sol#774)
  - _mint(msg.sender,realAmount) (Token.sol#538)
  - totalSupply_ += amount (Token.sol#773)
Reentrancy in Token.init(address,uint8,string,string,address,address,address) (Token.sol#369-403):
  External calls:
  - pool_.underlying().totalSupply() (Token.sol#389)
  State variables written after the call(s):
  - decimals_ = _decimals (Token.sol#393)

```

```

- finalBalance = pool_.underlying().balanceOf(address(this)) (Token.sol#530)
- pool_.underlying().safeTransfer(address(pool_),realAmount) (Token.sol#535)
- pool_.addToPool(realAmount) (Token.sol#536)
Event emitted after the call(s):
- MintFluid(msg.sender,realAmount) (Token.sol#539)
- Transfer(address(0),account,amount) (Token.sol#775)
- _mint(msg.sender,realAmount) (Token.sol#538)
Reentrancy in Token.erc20Out(uint256) (Token.sol#543-549):
  External calls:
  - pool_.takeFromPool(amount) (Token.sol#546)
  - pool_.underlying().safeTransfer(msg.sender,amount) (Token.sol#547)
  Event emitted after the call(s):
  - BurnFluid(msg.sender,amount) (Token.sol#548)
Reentrancy in Token.manualReward(address,uint256,address,uint256,uint256,uint256,bytes) (Token.sol#617-658):
  External calls:
  - require(bool,string)(rewardPoolAmount() >= winAmount,reward pool empty) (Token.sol#655)
  - totalAmount = pool_.totalPoolAmount() (Token.sol#552)
  Event emitted after the call(s):
  - BlockedReward(winner,amount,firstBlock,lastBlock) (Token.sol#562)
  - rewardFromPool(firstBlock,lastBlock,winner,winAmount) (Token.sol#657)
  - Reward(winner,amount,firstBlock,lastBlock) (Token.sol#575)
  - rewardFromPool(firstBlock,lastBlock,winner,winAmount) (Token.sol#657)
  - Transfer(address(0),account,amount) (Token.sol#775)
  - rewardFromPool(firstBlock,lastBlock,winner,winAmount) (Token.sol#657)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3

```

```

Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Address.verifyCallResult(bool,bytes,string) (Token.sol#107-125) uses assembly
- INLINE ASM (Token.sol#117-120)
Token.recover(bytes32,bytes) (Token.sol#660-686) uses assembly
- INLINE ASM (Token.sol#668-672)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Address.functionCall(address,bytes) (Token.sol#56-58) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256) (Token.sol#68-74) is never used and should be removed
Address.functionStaticCall(address,bytes) (Token.sol#89-91) is never used and should be removed
Address.functionStaticCall(address,bytes,string) (Token.sol#93-102) is never used and should be removed
Address.sendValue(address,uint256) (Token.sol#49-54) is never used and should be removed
SafeERC20.safeApprove(IERC20,address,uint256) (Token.sol#231-241) is never used and should be removed
SafeERC20.safeDecreaseAllowance(IERC20,address,uint256) (Token.sol#252-263) is never used and should be removed
SafeERC20.safeIncreaseAllowance(IERC20,address,uint256) (Token.sol#243-250) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code
INFO:Detectors:
Pragma version0.8.4 (Token.sol#2) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Low level call in Address.sendValue(address,uint256) (Token.sol#49-54):
- (success) = recipient.call{value: amount}() (Token.sol#52)
Low level call in Address.functionCallWithValue(address,bytes,uint256,string) (Token.sol#76-87):
- (success,returndata) = target.call{value: value}(data) (Token.sol#85)
Low level call in Address.functionStaticCall(address,bytes,string) (Token.sol#93-102):
- (success,returndata) = target.staticcall(data) (Token.sol#100)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Detectors:
Parameter WorkerConfig.init(address,address)._operator (Token.sol#151) is not in mixedCase
Parameter WorkerConfig.init(address,address)._emergencyCouncil (Token.sol#152) is not in mixedCase
Parameter Token.init(address,uint8,string,string,address,address,address)._liquidityProvider (Token.sol#370) is not in mixedCase
Parameter Token.init(address,uint8,string,string,address,address,address)._decimals (Token.sol#371) is not in mixedCase
Parameter Token.init(address,uint8,string,string,address,address,address)._name (Token.sol#372) is not in mixedCase
Parameter Token.init(address,uint8,string,string,address,address,address)._symbol (Token.sol#373) is not in mixedCase

Parameter Token.init(address,uint8,string,string,address,address,address)._emergencyCouncil (Token.sol#374) is not in mixedCase
Parameter Token.init(address,uint8,string,string,address,address,address)._operator (Token.sol#375) is not in mixedCase
Parameter Token.init(address,uint8,string,string,address,address,address)._workerConfig (Token.sol#376) is not in mixedCase
Parameter Token.setRestrictions(uint256,bool,uint256,uint256)._maxUncheckedReward (Token.sol#406) is not in mixedCase
Parameter Token.setRestrictions(uint256,bool,uint256,uint256)._mintLimitsEnabled (Token.sol#407) is not in mixedCase
Parameter Token.setRestrictions(uint256,bool,uint256,uint256)._globalMint (Token.sol#408) is not in mixedCase
Parameter Token.setRestrictions(uint256,bool,uint256,uint256)._userMint (Token.sol#409) is not in mixedCase
Parameter Token.disableEmergencyMode(address)._workerConfig (Token.sol#457) is not in mixedCase
Parameter Token.updateWorkerConfig(address)._workerConfig (Token.sol#467) is not in mixedCase
Parameter Token.updateRewardQuarantineThreshold(uint256)._maxUncheckedReward (Token.sol#475) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Variable WorkerConfig.init(address,address)._emergencyCouncil (Token.sol#152) is too similar to WorkerConfig.emergencyCouncil_ (Token.sol#144)
Variable Token.init(address,uint8,string,string,address,address,address)._emergencyCouncil (Token.sol#374) is too similar to Token.emergencyCouncil_ (Token.sol#343)
Variable Token.updateRewardQuarantineThreshold(uint256)._maxUncheckedReward (Token.sol#475) is too similar to Token.maxUncheckedReward_ (Token.sol#353)
Variable Token.setRestrictions(uint256,bool,uint256,uint256)._maxUncheckedReward (Token.sol#406) is too similar to Token.maxUncheckedReward_ (Token.sol#353)
Variable Token.setRestrictions(uint256,bool,uint256,uint256)._mintLimitsEnabled (Token.sol#407) is too similar to Token.mintLimitsEnabled_ (Token.sol#357)
Variable Token.updateWorkerConfig(address)._workerConfig (Token.sol#467) is too similar to Token.workerConfig_ (Token.sol#341)
Variable Token.init(address,uint8,string,string,address,address,address)._workerConfig (Token.sol#376) is too similar to Token.workerConfig_ (Token.sol#341)
Variable Token.disableEmergencyMode(address)._workerConfig (Token.sol#457) is too similar to Token.workerConfig_ (Token.sol#341)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#variable-names-are-too-similar
INFO:Detectors:
Token.slitherConstructorConstantVariables() (Token.sol#280-817) uses literals with too many digits:
- DEFAULT_GLOBAL_MINT_LIMIT = 1000000 (Token.sol#288)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#too-many-digits
INFO:Detectors:
init(address,address) should be declared external:
- WorkerConfig.init(address,address) (Token.sol#150-162)
updateOracles(WorkerConfig.OracleUpdate[]) should be declared external:
- WorkerConfig.updateOracles(WorkerConfig.OracleUpdate[]) (Token.sol#168-179)
getWorkerAddress(address) should be declared external:

getWorkerAddress(address) should be declared external:
- WorkerConfig.getWorkerAddress(address) (Token.sol#181-185)
getWorkerAddress() should be declared external:
- WorkerConfig.getWorkerAddress() (Token.sol#187-191)
enableEmergencyMode() should be declared external:
- WorkerConfig.enableEmergencyMode() (Token.sol#193-199)
disableEmergencyMode() should be declared external:
- WorkerConfig.disableEmergencyMode() (Token.sol#201-207)
init(address,uint8,string,string,address,address,address) should be declared external:
- Token.init(address,uint8,string,string,address,address,address) (Token.sol#369-403)
setRestrictions(uint256,bool,uint256,uint256) should be declared external:
- Token.setRestrictions(uint256,bool,uint256,uint256) (Token.sol#405-423)
updateOperator(address) should be declared external:
- Token.updateOperator(address) (Token.sol#425-431)
enableEmergencyMode() should be declared external:
- Token.enableEmergencyMode() (Token.sol#442-455)
disableEmergencyMode(address) should be declared external:
- Token.disableEmergencyMode(address) (Token.sol#457-465)
updateRewardQuarantineThreshold(uint256) should be declared external:
- Token.updateRewardQuarantineThreshold(uint256) (Token.sol#475-481)
updateMintLimits(uint256,uint256) should be declared external:
- Token.updateMintLimits(uint256,uint256) (Token.sol#483-493)
enableMintLimits(bool) should be declared external:
- Token.enableMintLimits(bool) (Token.sol#495-501)
erc20In(uint256) should be declared external:
- Token.erc20In(uint256) (Token.sol#503-541)
erc20Out(uint256) should be declared external:
- Token.erc20Out(uint256) (Token.sol#543-549)

```



```

batchReward(Winner[],uint256,uint256) should be declared external:
- Token.batchReward(Winner[],uint256,uint256) (Token.sol#580-601)
unlockReward(address,uint256,bool,uint256,uint256) should be declared external:
- Token.unlockReward(address,uint256,bool,uint256,uint256) (Token.sol#603-615)
name() should be declared external:
- Token.name() (Token.sol#689)
symbol() should be declared external:
- Token.symbol() (Token.sol#690)
decimals() should be declared external:
- Token.decimals() (Token.sol#691)
balanceOf(address) should be declared external:
- Token.balanceOf(address) (Token.sol#693-695)
transfer(address,uint256) should be declared external:
- Token.transfer(address,uint256) (Token.sol#697-700)
approve(address,uint256) should be declared external:
- Token.approve(address,uint256) (Token.sol#706-709)
transferFrom(address,address,uint256) should be declared external:
- Token.transferFrom(address,address,uint256) (Token.sol#711-719)
increaseAllowance(address,uint256) should be declared external:
- Token.increaseAllowance(address,uint256) (Token.sol#737-740)
decreaseAllowance(address,uint256) should be declared external:
- Token.decreaseAllowance(address,uint256) (Token.sol#742-750)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#public-function-that-could-be-declared-external
INFO:Slither:Token.sol analyzed (7 contracts with 75 detectors), 84 result(s) found
INFO:Slither:Use https://crytic.io/ to get access to additional detectors and Github integration

```

Slither log >> WorkerConfig.sol

```

INFO:Detectors:
WorkerConfig.init(address,address)._operator (WorkerConfig.sol#262) lacks a zero-check on :
- operator = _operator (WorkerConfig.sol#268)
WorkerConfig.init(address,address)._emergencyCouncil (WorkerConfig.sol#263) lacks a zero-check on :
- emergencyCouncil = _emergencyCouncil (WorkerConfig.sol#270)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Address.verifyCallResult(bool,bytes,string) (WorkerConfig.sol#203-223) uses assembly
- INLINE_ASM (WorkerConfig.sol#215-218)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Address.functionCall(address,bytes) (WorkerConfig.sol#87-89) is never used and should be removed
Address.functionCall(address,bytes,string) (WorkerConfig.sol#97-103) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256) (WorkerConfig.sol#116-122) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256,string) (WorkerConfig.sol#130-141) is never used and should be removed
Address.functionStaticCall(address,bytes) (WorkerConfig.sol#149-151) is never used and should be removed
Address.functionStaticCall(address,bytes,string) (WorkerConfig.sol#159-168) is never used and should be removed
Address.isContract(address) (WorkerConfig.sol#38-44) is never used and should be removed
Address.sendValue(address,uint256) (WorkerConfig.sol#62-67) is never used and should be removed
Address.verifyCallResult(bool,bytes,string) (WorkerConfig.sol#203-223) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code
INFO:Detectors:
Pragma version0.8.4 (WorkerConfig.sol#7) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Low level call in Address.sendValue(address,uint256) (WorkerConfig.sol#62-67):
- (success) = recipient.call{value: amount}() (WorkerConfig.sol#65)
Low level call in Address.functionCallWithValue(address,bytes,uint256,string) (WorkerConfig.sol#130-141):
- (success,returndata) = target.call{value: value}(data) (WorkerConfig.sol#139)
Low level call in Address.functionStaticCall(address,bytes,string) (WorkerConfig.sol#159-168):
- (success,returndata) = target.staticcall(data) (WorkerConfig.sol#166)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

```

```

INFO:Detectors:
Parameter WorkerConfig.init(address,address)._operator (WorkerConfig.sol#262) is not in mixedCase
Parameter WorkerConfig.init(address,address)._emergencyCouncil (WorkerConfig.sol#263) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Variable WorkerConfig.init(address,address)._emergencyCouncil (WorkerConfig.sol#263) is too similar to WorkerConfig.emergencyCouncil (WorkerConfig.sol#247)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#variable-names-are-too-similar
INFO:Detectors:
init(address,address) should be declared external:
- WorkerConfig.init(address,address) (WorkerConfig.sol#261-273)
updateOracles(WorkerConfig.OracleUpdate[]) should be declared external:
- WorkerConfig.updateOracles(WorkerConfig.OracleUpdate[]) (WorkerConfig.sol#280-291)
getWorkerAddress(address) should be declared external:
- WorkerConfig.getWorkerAddress(address) (WorkerConfig.sol#293-297)
getWorkerAddress() should be declared external:
- WorkerConfig.getWorkerAddress() (WorkerConfig.sol#299-303)
enableEmergencyMode() should be declared external:
- WorkerConfig.enableEmergencyMode() (WorkerConfig.sol#305-311)
disableEmergencyMode() should be declared external:
- WorkerConfig.disableEmergencyMode() (WorkerConfig.sol#317-323)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#public-function-that-could-be-declared-external
INFO:Slither:WorkerConfig.sol analyzed (2 contracts with 75 detectors), 26 result(s) found
INFO:Slither:Use https://crytic.io/ to get access to additional detectors and Github integration

```

Solidity Static Analysis

AaveLiquidityProvider.sol

Security

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in `SafeERC20.safeDecreaseAllowance(contract IERC20,address,uint256)`: Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 386:4:

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 323:16:

Gas & Economy

Gas costs:

Gas requirement of function `AaveLiquidityProvider.init` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 476:4:

Gas costs:

Gas requirement of function `AaveLiquidityProvider.takeFromPool` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 502:4:

Gas costs:

Gas requirement of function `AaveLiquidityProvider.totalPoolAmount` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 512:4:

Miscellaneous

Constant/View/Pure functions:

SafeERC20._callOptionalReturn(contract IERC20,bytes) : Potentially should be constant/view/pure but is not.

[more](#)

Pos: 405:4:

Constant/View/Pure functions:

LendingPoolInterface.withdraw(address,uint256,address) : Potentially should be constant/view/pure but is not.

[more](#)

Pos: 450:4:

Similar variable names:

AaveLiquidityProvider.init(address,address,address) : Variables have very similar names "owner" and "owner_".

Pos: 484:8:

Similar variable names:

AaveLiquidityProvider.init(address,address,address) : Variables have very similar names "owner" and "owner_".

Pos: 484:17:

No return:

ATokenInterface.balanceOf(address): Defines a return type but never explicitly returns a value.

Pos: 443:4:

No return:

LendingPoolInterface.withdraw(address,uint256,address): Defines a return type but never explicitly returns a value.

Pos: 450:4:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 494:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 503:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 507:8:

CompoundLiquidityProvider.sol**Security****Check-effects-interaction:**

Potential violation of Checks-Effects-Interaction pattern in SafeERC20.safeDecreaseAllowance(contract IERC20,address,uint256): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 385:4:

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in CompoundLiquidityProvider.init(address,address): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 477:4:

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 322:16:

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 246:50:

Gas & Economy

Gas costs:

Gas requirement of function `CompoundLiquidityProvider.init` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 477:4:

Gas costs:

Gas requirement of function `CompoundLiquidityProvider.addToPool` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 493:4:

Miscellaneous

Constant/View/Pure functions:

`CErc20Interface.accrueInterest()` : Potentially should be constant/view/pure but is not.

[more](#)

Pos: 451:4:

Constant/View/Pure functions:

`CErc20Interface.seize(address,address,uint256)` : Potentially should be constant/view/pure but is not.

[more](#)

Pos: 452:4:

Similar variable names:

`CompoundLiquidityProvider.init(address,address)` : Variables have very similar names "owner" and "owner_".

Pos: 484:17:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 505:8:

Token.sol**Security****Check-effects-interaction:**

Potential violation of Checks-Effects-Interaction pattern in Token.erc20In(uint256): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 851:4:

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in Token.erc20Out(uint256): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 905:4:

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 1086:8:

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 258:50:

Gas & Economy**Gas costs:**

Gas requirement of function Token.transferWithBeneficiary is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 1153:7:

Gas costs:

Gas requirement of function Token.decreaseAllowance is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 1174:7:

For loop over dynamic array:

Loops that do not have a fixed number of iterations, for example, loops that depend on storage values, have to be used carefully. Due to the block gas limit, transactions can only consume a certain amount of gas. The number of iterations in a loop can grow beyond the block gas limit which can cause the complete contract to be stalled at a certain point. Additionally, using unbounded loops incurs in a lot of avoidable gas costs. Carefully test how many items at maximum you can pass to such functions to make it successful.

[more](#)

Pos: 403:8:

For loop over dynamic array:

Loops that do not have a fixed number of iterations, for example, loops that depend on storage values, have to be used carefully. Due to the block gas limit, transactions can only consume a certain amount of gas. The number of iterations in a loop can grow beyond the block gas limit which can cause the complete contract to be stalled at a certain point. Additionally, using unbounded loops incurs in a lot of avoidable gas costs. Carefully test how many items at maximum you can pass to such functions to make it successful.

[more](#)

Pos: 973:8:

Miscellaneous

Constant/View/Pure functions:

SafeERC20._callOptionalReturn(contract IERC20,bytes) : Potentially should be constant/view/pure but is not.

[more](#)

Pos: 516:4:

Constant/View/Pure functions:

Token.recover(bytes32,bytes) : Is constant but potentially should not be.

[more](#)

Pos: 1075:4:

Similar variable names:

Token._burn(address,uint256) : Variables have very similar names "account" and "amount".

Pos: 1220:25:

Similar variable names:

Token._burn(address,uint256) : Variables have very similar names "account" and "amount".

Pos: 1220:46:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 923:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 936:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 1229:11:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 1243:15:

WorkerConfig.sol**Security****Check-effects-interaction:**

Potential violation of Checks-Effects-Interaction pattern in Address.functionCallWithValue(address,bytes,uint256,string): Could potentially lead to re-entrancy vulnerability.

[more](#)

Pos: 130:4:

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 215:16:

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 139:50:

Gas & Economy**Gas costs:**

Gas requirement of function WorkerConfig.init is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 261:4:

Gas costs:

Gas requirement of function WorkerConfig.updateOracles is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 280:4:

For loop over dynamic array:

Loops that do not have a fixed number of iterations, for example, loops that depend on storage values, have to be used carefully. Due to the block gas limit, transactions can only consume a certain amount of gas. The number of iterations in a loop can grow beyond the block gas limit which can cause the complete contract to be stalled at a certain point. Additionally, using unbounded loops incurs in a lot of avoidable gas costs. Carefully test how many items at maximum you can pass to such functions to make it successful.

[more](#)

Pos: 284:8:

Miscellaneous

Constant/View/Pure functions:

`Address.verifyCallResult(bool,bytes,string)` : Is constant but potentially should not be.

[more](#)

Pos: 203:4:

Similar variable names:

`WorkerConfig.updateOracles(struct WorkerConfig.OracleUpdate[])` : Variables have very similar names "oracles_" and "oracle".

Pos: 285:12:

Similar variable names:

`WorkerConfig.updateOracles(struct WorkerConfig.OracleUpdate[])` : Variables have very similar names "oracles_" and "oracle".

Pos: 289:21:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 300:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 307:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 318:8:

Solhint Linter

AaveLiquidityProvider.sol

```
AaveLiquidityProvider.sol:391:18: Error: Parse error: missing ';' at '{'
```

CompoundLiquidityProvider.sol

```
CompoundLiquidityProvider.sol:390:18: Error: Parse error: missing ';' at '{'
```

Token.sol

```
Token.sol:502:18: Error: Parse error: missing ';' at '{'  
Token.sol:1177:18: Error: Parse error: missing ';' at '{'  
Token.sol:1194:18: Error: Parse error: missing ';' at '{'  
Token.sol:1215:18: Error: Parse error: missing ';' at '{'  
Token.sol:1244:22: Error: Parse error: missing ';' at '{'
```

WorkerConfig.sol

```
WorkerConfig.sol:7:1: Error: Compiler version 0.8.11 does not satisfy the r semver requirement  
WorkerConfig.sol:8:1: Error: Compiler version v2 does not satisfy the r semver requirement  
WorkerConfig.sol:65:28: Error: Avoid using low level calls.  
WorkerConfig.sol:139:51: Error: Avoid using low level calls.  
WorkerConfig.sol:215:17: Error: Avoid using inline assembly. It is acceptable only in rare cases
```

Software analysis result:

All results have been reviewed and can be safely ignored.

#Hashlock.



#Hashlock.

Hashlock Pty Ltd